SOFTWARE ARCHITECTURE · ARCHITECTURAL KATA

GYM IT System

“The Tournament Arena” — a hands-on architectural kata: from problem and driving characteristics to style, decisions, trade-offs and risks.

Course	Software Architectures
Kata	Architectural Kata (Ted Neward format) applied to the GYM IT System
Reference system	github.com/personnna/software-architecture (microservices)
Date	June 2026

Team

Team member	Core domain	Architectural responsibility
Yeldana Kadenova	Challenges & leaderboards	Simplicity & UI architecture
Daniil Glazunov	User management — auth, profiles, roles	Security — JWT, AES-256, RBAC
Shattyk Kuziyeva	Tournament engine — brackets & scoring	Scalability & DevOps
Mirgalil Azhmukhambetov	Reporting & notifications	Fault tolerance & data

Contents

1 What an architectural kata is

An **architectural kata** is a practice exercise created by Ted Neward and popularised by Neal Ford and Mark Richards. The name comes from martial arts, where a *kata* is a rehearsed drill repeated to build instinct. In an architectural kata a team is given a short, deliberately incomplete business problem and a fixed amount of time to design an architecture for it, after which the team presents and **defends** its decisions to a review panel that asks hard questions.

The value is not a single "correct" answer but the reasoning: identifying what really matters, choosing a style to fit, and being explicit about trade-offs and risks. This document applies the kata method to the GYM IT System and follows the standard kata workflow:

1. State the problem in the canonical kata form — description, users, requirements and additional context.
2. Derive the **architectural characteristics** (the "-ilities") and select the few that drive the design.
3. Choose an **architecture style** that satisfies the driving characteristics, and justify it against alternatives.
4. Express the architecture in diagrams (context, containers, components, event flow).
5. Record the key decisions as **Architecture Decision Records (ADRs)**.
6. Analyse **trade-offs** and run a **risk-storming** pass with a risk matrix and mitigations.
7. Show how each driving characteristic is satisfied, and anticipate the review panel's questions.

> **How to read this document**
>
> Sections 2 is the kata **brief** (the problem). Sections 3–9 are the team's **response** — the design and its justification, grounded in the implemented reference system.

2 The kata — "The Tournament Arena"

2.1 Description

A growing network of fitness gyms wants a single web platform to run member competitions. Staff create tournaments for different disciplines, register competitors, and generate knock-out brackets. During an event, trainers record match results; the platform advances winners, keeps every member's record and produces a live points-based leaderboard. Members sign in to follow brackets and standings and to manage their own profile. The gym also wants to notify members when their matches are scheduled and when results are in. The business expects the platform to grow with the network and to stay available during busy tournament days, while protecting member accounts and personal data.

2.2 Users

- **Members** — the largest group; browse tournaments, brackets and the leaderboard, and edit their profile.
- **Trainers** — create and run tournaments, register participants and record scores.
- **Administrators** — manage user accounts and roles across the gym network.
- **Scale expectation** — thousands of registered members across the network, with sharp spikes of hundreds of concurrent users on tournament days; the platform should absorb roughly a 5× surge without redesign.

2.3 Functional requirements

- Self-registration and authentication with three roles (administrator, trainer, member).
- Create tournaments; register participants; generate single-elimination brackets.
- Record match results; reject ties; advance winners automatically.
- Maintain per-member statistics (played, wins, losses, points, titles) and a leaderboard.
- Self-service profiles; administrative user and role management.

- Notify members of scheduled matches and recorded results.

2.4 Additional context and constraints

- **Spiky load.** Traffic is uneven — quiet between events, intense during tournaments; scoring must stay responsive under load.
- **Sensitive data.** The system holds credentials and personal data (e.g. phone numbers) that must be protected at rest and in transit.
- **Independent evolution.** Different domains (accounts, tournaments, notifications) are built and owned by different people and should evolve and deploy independently.
- **Notifications must not block scoring.** A problem in the notification path must never stop a trainer from recording a result.
- **Runs anywhere.** The platform must build and run reproducibly via containers, locally and in the cloud.

3 Architectural characteristics

From the requirements and the additional context we derive the candidate architectural characteristics, then select the few that actually **drive** the design. Following kata practice, the driving set is kept deliberately small (the “rule of three”) so the architecture has a clear focus.

3.1 Driving characteristics worksheet

Characteristic	Driver?	Why it matters here
Scalability / Elasticity	DRIVER	Spiky tournament-day load; must scale the busy domains on demand without redesign.
Security	DRIVER	Credentials, roles and personal data (phone) must be protected; clear authn/authz boundary required.
Fault tolerance / Availability	DRIVER	A failure in notifications or one domain must not bring down scoring; stay up during events.
Maintainability / Deployability	Secondary	Domains owned by different people should evolve and deploy independently.
Simplicity (for clients)	Secondary	Clients should see one entry point and not the internal topology.
Performance	Considered	Scoring must feel responsive, but no sub-100 ms or high-frequency-trading needs.
Interoperability / Evolvability	Considered	REST + events keep integration open, but no third-party integration is required yet.

> **The three driving characteristics**
>
> **1. Scalability/Elasticity 2. Security 3. Fault tolerance/Availability.**
> Everything that follows — the style, the decisions, the risks — is judged primarily against these three.

3.2 Why not the others

Raw performance is a comfort, not a driver: tournament scoring is not latency-critical at the millisecond level. Simplicity and deployability matter, but they are **supporting** goals served by the same choices that deliver the top three (a gateway, independent services). Naming the drivers explicitly prevents “gold-plating” the architecture for characteristics the business did not ask for.

4 Architecture style decision

4.1 Options considered

Style	Fit against the drivers
Single-layer monolith	Simple to start, but the whole app scales and fails as one unit — weak on scalability and fault isolation.
Modular monolith	Clear internal boundaries and easy to run, but still one deployment and one database — cannot scale or deploy domains independently. (Built as the baseline — see note.)
Microservices	Independent scaling, deployment and fault isolation per domain — directly matches all three drivers, at the cost of operational complexity.
Event-driven (pure)	Excellent decoupling, but a fully asynchronous core is overkill for request/response scoring and harder to reason about.

4.2 Decision

The team selected a **microservices architecture with event-driven integration** for the asynchronous parts (notifications). This is the only option that satisfies all three driving characteristics at once: busy domains scale independently (scalability), a single gateway provides one security boundary while services stay internal (security), and domains fail in isolation with notifications decoupled from scoring (fault tolerance). Synchronous REST is kept for the request/response interactions where it is the simplest correct choice, and asynchronous events are used only where decoupling pays off.

> **Grounded in a real comparison**
>
> The same domain was also built as a **modular monolith** (repository `Ilagrim2/Gymonolith`). That baseline confirmed the trade-off: it was faster to build and simpler to run, but could not scale or deploy domains independently — which is exactly why the microservices version exists. The kata's decision is therefore evidence-based, not theoretical.

5 The architecture

The design is expressed with the C4 model — context, then containers, then a component view for the security-critical service — plus the layered overview and the asynchronous event flow.

5.1 System context

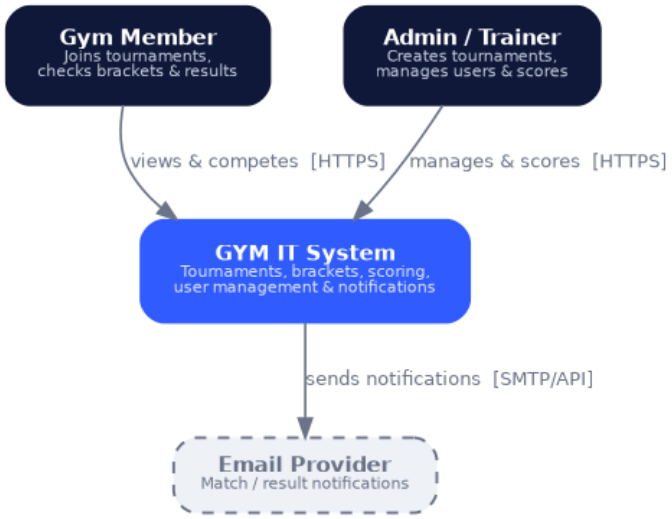

Figure 1 — System context: who uses the platform and the external notification provider.

5.2 Architecture overview

The platform is layered into a deployment plane, an API layer, the message broker, the microservices and their databases:

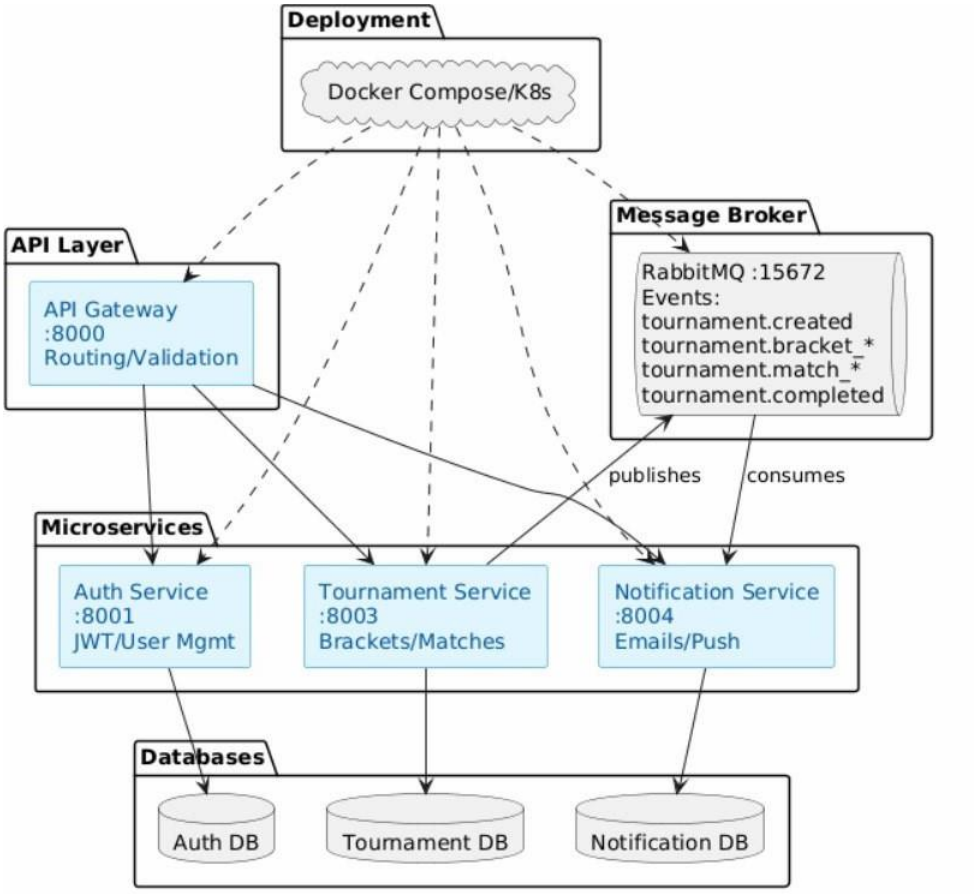

Figure 2 — Layered architecture overview.

5.3 Containers

Clients reach only the API Gateway (port 8000), which routes to the Auth service (8001) and Tournament service (8003) and serves the frontend. The Tournament service publishes events to RabbitMQ, consumed by

the Notification service (8004), and updates player statistics in the Auth service with a service token.

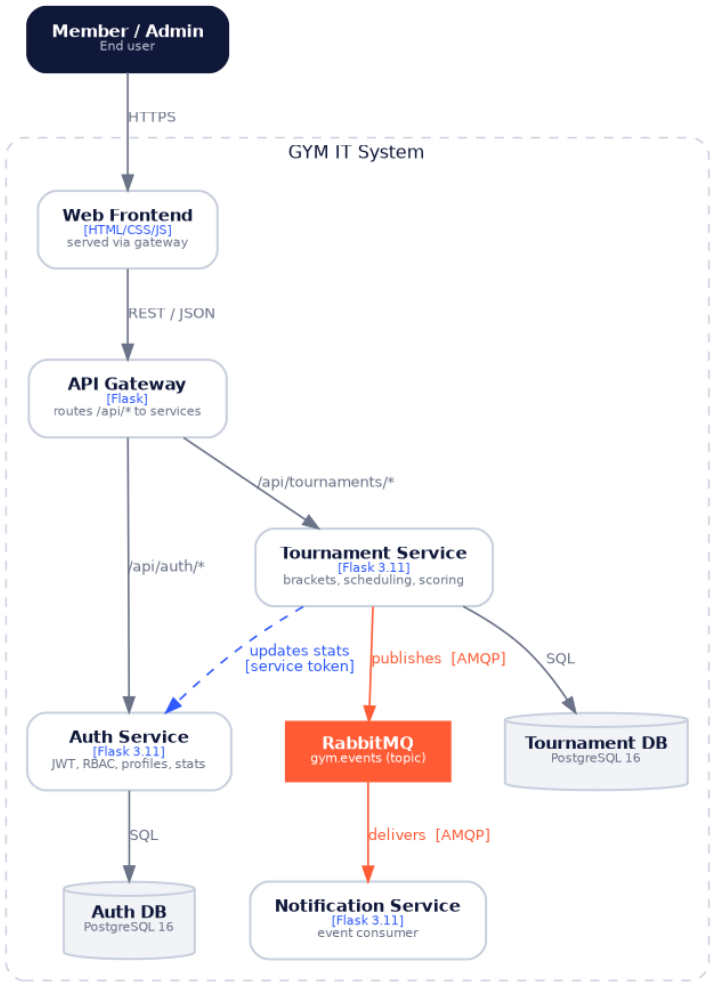

Figure 3 — Container diagram (C4 level 2).

5.4 Security component view

Because **security** is a driver, the Auth service is shown at component level: route handlers are guarded by RBAC middleware that verifies JWTs, a crypto helper encrypts sensitive fields with AES-256-GCM, and ORM models persist to the service's own database.

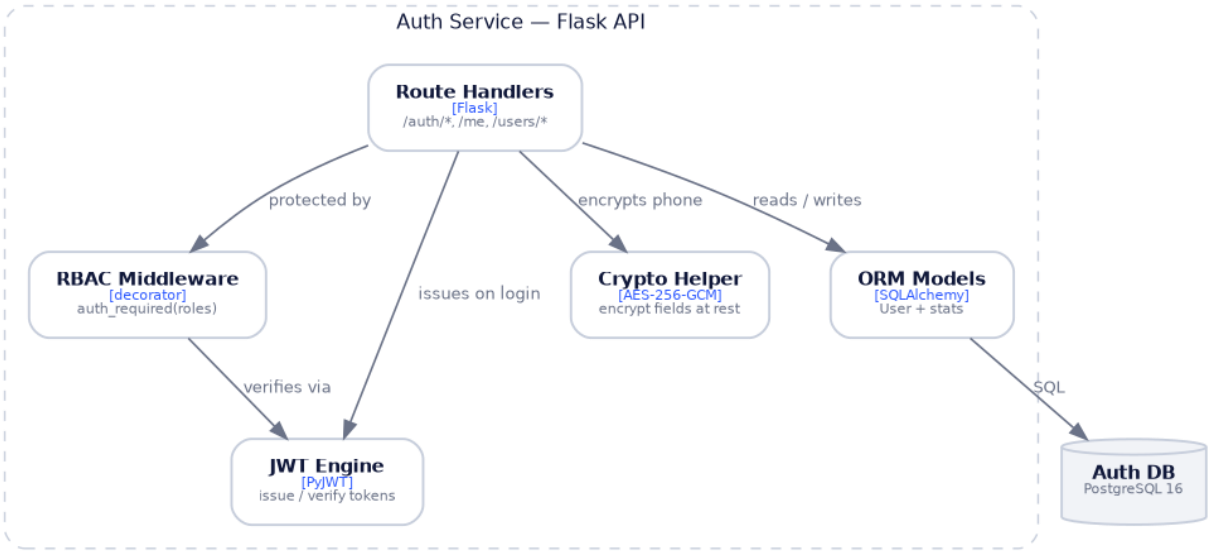

Figure 4 — Auth service components (C4 level 3).

5.5 Asynchronous event flow

The Tournament service publishes domain events to the durable `gym.events` topic exchange after each commit; the Notification service consumes them from a durable queue. Events: `tournament.created`, `participant_added`, `bracket_generated`, `match_scheduled`, `match_result_recorded`, `completed`.

Figure 5 — Asynchronous event flow through RabbitMQ.

6 Architecture decision records

The key decisions are recorded as ADRs so the reasoning — not just the outcome — is preserved.

ADR-001 Adopt a microservices architecture	**Accepted**
Context	Domains must scale, deploy and fail independently (drivers: scalability, fault tolerance), and are owned by different people.
Decision	Split the system into independent Flask services (gateway, auth, tournament, notification), each owning its data.
Consequences	+ Independent scaling, deployment and fault isolation; clear ownership. – Operational complexity and distributed-data concerns (addressed by ADR-003/004).

ADR-002 API Gateway as the single entry point	Accepted
Context	Clients should not know the internal topology, and routing, CORS and TLS need one consistent place (drivers: security, simplicity).
Decision	Introduce a Flask API Gateway that routes /api/* to services and serves the frontend; backend services stay on the internal network.
Consequences	+ One security boundary and a simple client contract. – The gateway is critical; mitigated with replicas behind a load balancer (ADR-007).

ADR-003 Database per service	Accepted
Context	Strict decoupling and independent schema evolution are required (drivers: maintainability, fault tolerance).
Decision	Give each data-owning service its own PostgreSQL database; no cross-service table access — data is shared only via APIs or events.
Consequences	+ Isolation and independent evolution. – No cross-service joins; some data is eventually consistent across services.

ADR-004 Asynchronous events via RabbitMQ	Accepted
Context	Notifications and reporting must not block or break scoring (driver: fault tolerance/availability).
Decision	Publish domain events to a durable topic exchange after each commit; the Notification service consumes from a durable queue.
Consequences	+ Scoring stays available even if notifications lag or the consumer is down. – Eventual consistency and a broker dependency to operate.

ADR-005 Stateless JWT with centralized RBAC	Accepted
Context	Authentication must work across stateless services with consistent authorization (driver: security).
Decision	Issue HS256 JWTs carrying identity and role; enforce role checks with a single reusable middleware on every protected endpoint.
Consequences	+ Stateless, uniform authorization shared by all services. – Immediate revocation is harder; mitigated with short token lifetimes.

ADR-006 AES-256-GCM for sensitive data at rest	Accepted
Context	Personal data (phone numbers) must be protected at rest (driver: security).
Decision	Encrypt sensitive fields with AES-256-GCM before storage; decrypt only for authorized profile/admin flows.
Consequences	+ Confidentiality plus tamper detection. – The team owns key management; keys come from secrets, never the repo.

ADR-007 Kubernetes with autoscaling and health probes	Accepted
Context	The platform must absorb tournament-day spikes and recover from failures automatically (drivers: scalability, availability).
Decision	Run services as Kubernetes Deployments with a Horizontal Pod Autoscaler (2–10 replicas at 70% CPU), liveness/readiness probes, a LoadBalancer and a TLS Ingress.
Consequences	+ Elastic scaling and self-healing restarts. – Additional infrastructure to configure and operate.

7 Trade-off analysis

Every decision buys a driver at a price. The major trade-offs:

Decision	What it buys	What it costs
Microservices	Independent scaling, deployment, fault isolation	Operational complexity; distributed data
API Gateway	Single security boundary; simple client	A critical component to keep highly available
Database per service	Decoupling; independent evolution	No cross-service joins; eventual consistency
Async events (RabbitMQ)	Scoring stays available; loose coupling	Eventual consistency; a broker to operate
Stateless JWT	Stateless, uniform authorization	Harder revocation (mitigated by short expiry)
Kubernetes + HPA	Elasticity; self-healing	Infrastructure and operational overhead

8 Risk storming

A risk-storming pass identifies where the architecture could fail and rates each risk by **probability × impact**. The matrix shows that the high-impact risks are deliberately held at low probability by the mitigations below — there are no red (high/high) cells.

8.1 Risk matrix

Impact ↓ / Probability →	Low	Medium	High
High	R2 · R3 · R6	—	—
Medium	R7	R1 · R4 · R5	—
Low	R8	—	—

Green = low Amber = medium Red = high. Cells list the risk IDs from the register below.

8.2 Risk register and mitigations

ID	Risk	Sev.	Mitigation
R1	RabbitMQ outage — events/notifications delayed or lost	Med	Publish only after commit; retry then log; core scoring path is unaffected; durable queue replays on recovery.
R2	Auth service unavailable — no logins or token checks	Med	Multiple replicas, health probes and autoscaling; stateless tokens keep verifying during brief blips.
R3	JWT signing-secret leakage — token forgery	Med	Secret stored in Kubernetes secrets, never in the repo; short token lifetime; rotation possible.
R4	Cross-service stat inconsistency after a result	Med	Synchronous service-token update with idempotent writes; future outbox pattern for guaranteed delivery.
R5	Operational complexity — misconfiguration on deploy	Med	Docker-Compose/Kubernetes parity, CI checks and health probes catch bad configurations early.
R6	API Gateway down — whole platform unreachable	Med	Three replicas behind a LoadBalancer; stateless and horizontally scalable.
R7	Per-service backup/restore complexity	Low	Standard PostgreSQL backups per database; isolation limits blast radius.
R8	Scoring race conditions under load	Low	Server-side validation, single-writer per match, ties rejected at the service.

9 How the kata is satisfied

Each driving characteristic maps to concrete mechanisms that exist in the reference system:

Driver	Mechanism	Evidence in the system
Scalability / Elasticity	Stateless services; Kubernetes HPA; gateway routing	k8s Deployments + HorizontalPodAutoscaler (2–10 @70% CPU); LoadBalancer
Security	JWT + centralized RBAC; AES-256-GCM; TLS; gateway boundary	auth-service middleware; encrypted phone field; TLS Ingress; internal-only services
Fault tolerance / Availability	Async events; DB-per-service; health probes; retries	RabbitMQ durable exchange/queue; /healthz probes; publish-after-commit with retry
Maintainability (support)	Independent services; CI; OpenAPI + C4 docs	Per-service tests; GitHub Actions pipeline; documented contracts
Simplicity (support)	Single API Gateway; consistent REST	One public entry point; uniform /api/* routing

9.1 Anticipated questions from the review panel

Isn't microservices over-engineering for a gym app? For a single small gym, yes — that is why a modular monolith was built first. The kata's brief is a growing *network* with tournament-day spikes and independent domain ownership, which are exactly the conditions microservices address.

What happens when RabbitMQ is down? Nothing on the scoring path: events publish only after the database commit, the publisher retries and then logs, and the durable queue replays to the Notification service when it recovers. Notifications are delayed, never a blocker.

How do you keep player statistics consistent across services? The Tournament service updates the Auth service synchronously with a service token using idempotent writes; for stronger guarantees the next step is a transactional outbox so a stats update can never be missed.

The gateway and the auth service look like single points of failure. Both are stateless and run as multiple replicas behind a load balancer with health probes, so Kubernetes reschedules them automatically; no user session state is lost because authentication is stateless.

How is token revocation handled? Tokens are short-lived, which bounds exposure; immediate revocation (a deny-list or rotating signing keys) is a documented future enhancement — an accepted trade-off of stateless JWT (ADR-005).